

# Guia Completa de OpenSCAD

Del Cubo al Robot Cuadrupe

Basada en el diseno del USS SpiderBot

Robot cuadrupe de 8 grados de libertad

Taller de Programacion I

Universidad San Sebastian

<https://github.com/moises-inc/USS-SPIDERBOT-solemne-3>

2026

# Índice general

|                                                                              |           |
|------------------------------------------------------------------------------|-----------|
| <b>Introduccion</b>                                                          | <b>4</b>  |
| <b>1. Fundamentos Geometricos</b>                                            | <b>6</b>  |
| 1.1. Formas primitivas 3D . . . . .                                          | 6         |
| 1.2. Sistema de coordenadas . . . . .                                        | 6         |
| 1.3. El parametro <b>center</b> . . . . .                                    | 7         |
| 1.4. Resolucion de curvas: <b>\$fn</b> , <b>\$fa</b> , <b>\$fs</b> . . . . . | 7         |
| 1.5. Caso Real: Primitivas en el SpiderBot . . . . .                         | 7         |
| <b>2. Transformaciones Geometricas</b>                                       | <b>9</b>  |
| 2.1. <b>translate([x, y, z])</b> . . . . .                                   | 9         |
| 2.2. <b>rotate([x, y, z])</b> . . . . .                                      | 9         |
| 2.3. <b>scale([x, y, z])</b> . . . . .                                       | 9         |
| 2.4. <b>mirror([x, y, z])</b> — Reflejo especular . . . . .                  | 10        |
| 2.5. <b>color("nombre")</b> — Color para preview . . . . .                   | 10        |
| 2.6. <b>resize([x, y, z])</b> — Redimensionar . . . . .                      | 10        |
| 2.7. Composicion: se lee de adentro hacia afuera . . . . .                   | 10        |
| 2.8. Caso Real: Rotaciones y traslaciones . . . . .                          | 10        |
| 2.9. Caso Real 2:Codigo de colores . . . . .                                 | 11        |
| <b>3. Operaciones Booleanas (El Corazon del Diseno)</b>                      | <b>12</b> |
| 3.1. <b>union()</b> . . . . .                                                | 12        |
| 3.2. <b>difference()</b> . . . . .                                           | 12        |
| 3.3. <b>intersection()</b> . . . . .                                         | 12        |
| 3.4. Anidamiento . . . . .                                                   | 13        |
| 3.5. Caso Real: El soporte de servo . . . . .                                | 13        |
| <b>4. Variables y Parametros</b>                                             | <b>15</b> |
| 4.1. Declaracion y asignacion . . . . .                                      | 15        |
| 4.2. Variables parametricas . . . . .                                        | 15        |
| 4.3. <b>let()</b> — Variables locales . . . . .                              | 15        |

|           |                                                                      |           |
|-----------|----------------------------------------------------------------------|-----------|
| 4.4.      | <code>assert()</code> — Validacion . . . . .                         | 15        |
| 4.5.      | <code>function</code> — Funciones definidas por el usuario . . . . . | 16        |
| 4.6.      | Funciones de string y lista . . . . .                                | 16        |
| 4.7.      | Operador ternario . . . . .                                          | 16        |
| 4.8.      | Constantes especiales . . . . .                                      | 16        |
| 4.9.      | Caso Real: Variables parametricas del SpiderBot . . . . .            | 16        |
| <b>5.</b> | <b>Modulos — Funciones del Diseno Parametrico</b>                    | <b>18</b> |
| 5.1.      | Sintaxis basica . . . . .                                            | 18        |
| 5.2.      | Ejemplo: tornillo parametrico . . . . .                              | 18        |
| 5.3.      | <code>children()</code> — Modulos que envuelven geometria . . . . .  | 18        |
| 5.4.      | DRY en CAD . . . . .                                                 | 19        |
| 5.5.      | Caso Real: Los modulos del SpiderBot . . . . .                       | 19        |
| <b>6.</b> | <b>Iteracion y Simetria — El Bucle <code>for</code></b>              | <b>20</b> |
| 6.1.      | Sintaxis . . . . .                                                   | 20        |
| 6.2.      | Distribucion radial . . . . .                                        | 20        |
| 6.3.      | <code>intersection_for()</code> . . . . .                            | 20        |
| 6.4.      | List Comprehensions . . . . .                                        | 20        |
| 6.5.      | Caso Real: Las 4 patas del SpiderBot . . . . .                       | 21        |
| <b>7.</b> | <b>Operaciones Avanzadas</b>                                         | <b>22</b> |
| 7.1.      | <code>hull()</code> — Envolverte convexa . . . . .                   | 22        |
| 7.2.      | <code>minkowski()</code> — Suma de Minkowski . . . . .               | 22        |
| 7.3.      | <code>linear_extrude()</code> — Extrusion de 2D a 3D . . . . .       | 22        |
| 7.4.      | <code>text()</code> — Texto 2D . . . . .                             | 22        |
| 7.5.      | <code>offset()</code> — Desplazamiento 2D . . . . .                  | 23        |
| 7.6.      | <code>projection()</code> — Proyeccion 3D a 2D . . . . .             | 23        |
| 7.7.      | <code>polygon()</code> — Perfiles 2D . . . . .                       | 23        |
| 7.8.      | <code>rotate_extrude()</code> — Revolucion . . . . .                 | 23        |
| 7.9.      | Caso Real: El femur con <code>hull()</code> . . . . .                | 23        |
| <b>8.</b> | <b>Archivos Multiples — <code>use</code> e <code>include</code></b>  | <b>25</b> |
| 8.1.      | <code>use &lt;archivo.scad&gt;</code> . . . . .                      | 25        |
| 8.2.      | <code>import()</code> — Importar STL/SVG/DXF . . . . .               | 25        |
| 8.3.      | <code>include &lt;archivo.scad&gt;</code> . . . . .                  | 25        |
| 8.4.      | Organizacion de proyectos . . . . .                                  | 25        |
| 8.5.      | Caso Real: El ensamble del SpiderBot . . . . .                       | 26        |
| <b>9.</b> | <b>Renderizado, Debug y Exportacion a STL</b>                        | <b>27</b> |
| 9.1.      | Preview (F5) vs Render (F6) . . . . .                                | 27        |
| 9.2.      | Modifier Characters — Debug visual . . . . .                         | 27        |

---

|                                                                       |           |
|-----------------------------------------------------------------------|-----------|
| 9.3. <code>render()</code> — Forzar CGAL en preview . . . . .         | 28        |
| 9.4. <code>\$t</code> y animacion . . . . .                           | 28        |
| 9.5. <code>\$preview</code> — Diferenciar preview de render . . . . . | 28        |
| 9.6. Exportar a STL (F7) . . . . .                                    | 28        |
| 9.7. CLI de <code>OpenSCAD</code> . . . . .                           | 28        |
| 9.8. Parametros de calidad . . . . .                                  | 28        |
| 9.9. Caso Real: Comandos CLI para el SpiderBot . . . . .              | 29        |
| <b>10. Proyecto Final — Replica el USS SpiderBot desde Cero</b>       | <b>30</b> |
| 10.1. Paso 1: El chasis circular . . . . .                            | 30        |
| 10.2. Paso 2: El soporte de servo parametrico . . . . .               | 30        |
| 10.3. Paso 3: Integrar soportes en el chasis . . . . .                | 31        |
| 10.4. Paso 4: El femur con <code>hull()</code> . . . . .              | 31        |
| 10.5. Paso 5: La tibia . . . . .                                      | 31        |
| 10.6. Paso 6: Ensamblar todo . . . . .                                | 31        |
| 10.7. Paso 7: Exportar a STL . . . . .                                | 31        |
| <b>Apéndice A: Hoja de Referencia Rapida</b>                          | <b>32</b> |
| <b>Apéndice B: Recursos y Documentacion Oficial</b>                   | <b>34</b> |

# Introduccion

**OpenSCAD** es un modelador CAD 3D basado exclusivamente en código (CSG — Constructive Solid Geometry). A diferencia de herramientas como Fusion 360 o TinkerCAD, aquí no arrastras mouse: escribes instrucciones y el programa las evalúa para generar geometría.

## Por que OpenSCAD para robotica?

- **Totalmente paramétrico:** cambias un número y todo el modelo se actualiza.
- **Versionable:** los archivos `.scad` son texto plano, ideales para Git.
- **Preciso:** las operaciones booleanas son exactas, no hay errores de manifold.
- **Gratuito y open-source.**
- **CLI headless:** puedes exportar STLs desde scripts.

## Instalacion

| Sistema | Comando / Enlace                                                                                      |
|---------|-------------------------------------------------------------------------------------------------------|
| Windows | Descargar installer desde <a href="https://openscad.org">https://openscad.org</a>                     |
| macOS   | <code>brew install openscad</code> o descargar .dmg                                                   |
| Linux   | <code>sudo apt install openscad</code> (Debian/Ubuntu)<br><code>sudo pacman -S openscad</code> (Arch) |

## Interfaz

Al abrir OpenSCAD ves tres paneles:

1. **Editor de código** (izquierda) — aquí escribes.
2. **Vista 3D** (derecha) — aquí ves el resultado.

3. **Consola** (abajo) — muestra errores, advertencias y outputs.

## Flujo de trabajo

1. Escribes código en el editor.
2. Presionas **F5** — *Preview* (vista rápida, usa OpenCSG).
3. Presionas **F6** — *Render* (cálculo CGAL completo, geometría exacta).
4. Archivo > Exportar > Exportar como STL (o **F7**) — obtienes el archivo para impresión 3D.

# Capítulo 1

## Fundamentos Geometricos

### 1.1 Formas primitivas 3D

OpenSCAD tiene cuatro formas solidas basicas:

```
1 // Cubo: cube([ancho, fondo, alto], center);
2 cube([10, 20, 30]);           // Esquina en el origen
3 cube([10, 20, 30], center=true); // Centrado en el origen
4
5 // Esfera: sphere(r);
6 sphere(r = 10);               // Radio 10mm
7
8 // Cilindro: cylinder(r, h);
9 cylinder(r = 5, h = 20);       // Radio 5mm, altura 20mm
10
11 // Cilindro con radios distintos (cono truncado):
12 cylinder(r1 = 5, r2 = 3, h = 10);
```

### 1.2 Sistema de coordenadas

OpenSCAD usa coordenadas cartesianas diestras:

- **Eje X:** horizontal (derecha = positivo)
- **Eje Y:** profundidad (atras = positivo)
- **Eje Z:** vertical (arriba = positivo)

## 1.3 El parametro center

Por defecto, `cube()`, `cylinder()` y `sphere()` se posicionan con una esquina o base en el origen.

```
1 // Sin center: el cubo crece desde (0,0,0) hacia (10,20,30)
2 cube([10, 20, 30]);
3
4 // Con center=true: el cubo se centra en (0,0,0)
5 cube([10, 20, 30], center=true);
```

## 1.4 Resolucion de curvas: \$fn, \$fa, \$fs

Las formas curvas se aproximan con poligonos:

```
1 // $fn controla el numero de fragmentos (lados)
2 cylinder(r = 10, h = 20, $fn = 6); // Hexagono - baja calidad
3 cylinder(r = 10, h = 20, $fn = 50); // Calidad media
4 cylinder(r = 10, h = 20, $fn = 100); // Muy suave - render lento
5
6 // Tambien se puede definir globalmente
7 $fn = 50; // Todas las curvas usaran 50 fragmentos
```

**Regla practica:** `$fn = 30` para preview rapido, `$fn = 50--80` para exportacion a STL.

## 1.5 Caso Real: Primitivas en el SpiderBot

En `cad/cuerpo_cuadruepdo.scad`, la placa base se define con `cylinder()`:

```
1 // Archivo: cad/cuerpo_cuadruepdo.scad
2 $fn = 50;
3
4 cuerpo_r = 65.0; // Radio del chasis circular
5 cuerpo_t = 3.0; // Grosor de las placas
6
7 module placa_base_cuadruepodo() {
8     difference() {
9         // Placa inferior solida: cilindro de 65mm de radio, 3mm de grosor
10        cylinder(r = cuerpo_r, h = cuerpo_t, center=true);
11        // Agujeros para los 4 pilares espaciadores M3
12        for (a = [0, 90, 180, 270]) {
13            rotate([0, 0, a])
14                translate([spacer_r, 0, 0])
15                    cylinder(r=1.6, h=cuerpo_t+2, center=true);
16        }
17    }
```



18 }

# Capítulo 2

## Transformaciones Geometricas

Las transformaciones mueven, rotan y escalan objetos. **No crean geometria nueva**, sino que modifican la existente.

### 2.1 `translate([x, y, z])`

Desplaza el objeto hijo en el espacio:

```
1 translate([20, 10, 5])  
2   cube([10, 10, 10]);
```

### 2.2 `rotate([x, y, z])`

Rota alrededor del origen en grados:

```
1 // Rota 45 grados alrededor de Z  
2 rotate([0, 0, 45])  
3   cube([20, 5, 5]);  
4 // Rota 90 grados alrededor de X (pone el cubo vertical)  
5 rotate([90, 0, 0])  
6   cube([20, 5, 5]);
```

### 2.3 `scale([x, y, z])`

Escala en cada eje:

```
1 // No uniforme: estira en X  
2 scale([2, 1, 1]) sphere(r = 10);  
3 // Uniforme: agranda todo al doble  
4 scale(2) sphere(r = 10);
```

## 2.4 mirror([x, y, z]) — Reflejo especular

Refleja el objeto como si lo miraras a traves de un plano:

```
1 // Reflejar en X (plano YZ)
2 mirror([1, 0, 0]) cube([10, 20, 5]);
3 // Reflejar en diagonal
4 mirror([1, 1, 0]) cube([10, 20, 5]);
```

## 2.5 color("nombre") — Color para preview

Asigna color a los hijos (solo Preview F5):

```
1 color("RoyalBlue") cube([10, 10, 10]);
2 color([1.0, 0.5, 0]) sphere(r = 5); // RGB [0-1]
3 color("LightGray", 0.5) cube([20, 5, 5]); // Semitransparente
```

## 2.6 resize([x, y, z]) — Redimensionar

Fuerza dimensiones absolutas, a diferencia de scale():

```
1 // Esfera de radio 10 -> elipsoide de 30x60x10
2 resize([30, 60, 10]) sphere(r = 10);
```

## 2.7 Composicion: se lee de adentro hacia afuera

```
1 translate([30, 0, 0])
2   rotate([0, 0, 45])
3     cube([10, 10, 10]);
```

## 2.8 Caso Real: Rotaciones y traslaciones

En cad/cuerpo\_cuadruepdo.scad:

```
1 for (a = [45, 135, 225, 315]) {
2   rotate([0, 0, a])
3     translate([cuerpo_r - 7, 0, cuerpo_t/2])
4       rotate([0, 0, -90])
5         soporte_servo_cadera();
6 }
```

## 2.9 Caso Real 2: Codigo de colores

En cad/ensamble\_cuadruepodo.scad:

```
1 module dummy_servo() {  
2     color("RoyalBlue") {  
3         cube([12.5, 23.0, 22.5], center=true);  
4     }  
5     color("White") {  
6         translate([0, 5.5, 14])  
7             cylinder(r=6, h=1.5, center=true);  
8     }  
9 }
```

## Capítulo 3

# Operaciones Booleanas (El Corazon del Diseno)

Las operaciones booleanas combinan solidos para crear formas complejas.

### 3.1 union()

Fusiona todos los hijos en un solo solido:

```
1 union() {  
2     cube([10, 10, 10]);  
3     translate([5, 5, 0])  
4         cylinder(r = 5, h = 10);  
5 }
```

### 3.2 difference()

Resta los hijos subsiguientes del primero:

```
1 difference() {  
2     cube([20, 20, 10], center=true);  
3     translate([0, 0, -1])  
4         cylinder(r = 3, h = 12);  
5 }
```

### 3.3 intersection()

Devuelve solo el volumen compartido:

```
1 intersection() {  
2     sphere(r = 10);  
3     cube([10, 10, 10], center=true);  
4 }
```

## 3.4 Anidamiento

Las booleanas se anidan sin limite:

```
1 difference() {  
2     union() {  
3         cylinder(r = 20, h = 5);  
4         translate([15, 0, 0])  
5             cube([10, 10, 5], center=true);  
6     }  
7     cylinder(r = 3, h = 10, center=true);  
8 }
```

## 3.5 Caso Real: El soporte de servo

El modulo soporte\_servo\_cadera() en cad/cuerpo\_cuadruepdo.scad:

```
1 module soporte_servo_cadera() {  
2     difference() {  
3         // SOLIDO BASE  
4         translate([0, 0, 7.5])  
5             cube([36, 28, 15], center=true);  
6         // 1. Cavidad dual-depth para servo SG90  
7         translate([0, 0, 2.9])  
8             cube([30.0, 24.2, 4.2], center=true);  
9         translate([0, 0, 9.5])  
10            cube([25.0, 24.2, 9.0], center=true);  
11        // 2. Ranura para bridas  
12        translate([0, 5.5, 7.5])  
13            cube([34.5, 3.2, 13.5], center=true);  
14        // 3. Agujeros pasantes para tornillos M2  
15        translate([-14.25, 0, 7.5])  
16            rotate([90, 0, 0])  
17                cylinder(r=1.0, h=35, center=true);  
18        translate([14.25, 0, 7.5])  
19            rotate([90, 0, 0])  
20                cylinder(r=1.0, h=35, center=true);  
21        // 4. Abertura para corona de salida  
22        translate([5.5, 12.0, 7.5])  
23            rotate([90, 0, 0])
```

```
24         cylinder(r=6.2, h=15, center=true);
25         // 5. Rebaje para rotacion del femur
26         translate([6.5, 12.0, 7.5])
27             cube([23, 5, 16], center=true);
28     }
29 }
```

**Dual-depth pocket:** la cavidad tiene dos anchos (30mm abajo para cables, 25mm arriba para rosca de tornillos M2).

# Capítulo 4

## Variables y Parametros

### 4.1 Declaracion y asignacion

En OpenSCAD las variables se asignan con = y son **inmutables**:

```
1 ancho = 20; profundidad = 30; alto = 10;  
2 cube([ancho, profundidad, alto]);
```

### 4.2 Variables parametricas

```
1 longitud = 50; ancho = 20; altura = 10;  
2 pared = 2.0; tornillo_r = 1.5;  
3  
4 difference() {  
5     cube([longitud, ancho, altura]);  
6     translate([pared, pared, -1])  
7         cube([longitud - 2*pared, ancho - 2*pared, altura + 2]);  
8 }
```

### 4.3 let() — Variables locales

```
1 let (ancho = 30, largo = ancho * 1.5) {  
2     cube([ancho, largo, 5]);  
3 }
```

### 4.4 assert() — Validacion



```

1 module soporte(alto, ancho) {
2     assert(alto > 0, "El alto debe ser positivo");
3     assert(ancho > 0, "El ancho debe ser positivo");
4     cube([ancho, ancho, alto]);
5 }

```

## 4.5 function — Funciones definidas por el usuario

```

1 function area_circulo(radio) = PI * radio * radio;
2 function volumen_cilindro(radio, alto) = area_circulo(radio) * alto;
3 echo(volumen_cilindro(10, 20)); // 6283.19

```

## 4.6 Funciones de string y lista

```

1 // str() - convertir a string
2 etiqueta = str("Pieza-", 3, ".stl"); // "Pieza-3.stl"
3 // concat() - unir listas
4 c = concat([1, 2, 3], [4, 5]); // [1, 2, 3, 4, 5]
5 // len() - longitud
6 largo = len([10, 20, 30]); // 3

```

## 4.7 Operador ternario

```

1 usar_agujeros = true;
2 resolucion = (usar_agujeros ? 50 : 20);

```

## 4.8 Constantes especiales

```

1 radio = 10;
2 circunferencia = 2 * PI * radio; // 62.8318
3 echo(circunferencia);

```

## 4.9 Caso Real: Variables parametricas del SpiderBot

En cad/eslabon\_pata\_cuadripedo.scad:

```

1 link_len = 55.0; // Longitud entre cadera y rodilla
2 link_h = 25.0; // Altura extremo de rodilla
3 link_t = 16.0; // Grosor bloque servo rodilla
4 servo_w = 23.0; servo_t = 12.5;
5 servo_h = 22.5; ear_dist = 28.5;

```

Si cambias `link_len = 55` a `70`, el bolsillo del servo y la abertura del engranaje se desplazan automaticamente.

# Capítulo 5

## Modulos — Funciones del Diseno Parametrico

### 5.1 Sintaxis basica

```
1 module nombre(param1, param2 = valor_defecto) {  
2     // Geometria aqui  
3 }
```

### 5.2 Ejemplo: tornillo parametrico

```
1 module tornillo(longitud, diametro = 4) {  
2     radio = diametro / 2;  
3     cylinder(r = radio, h = longitud);  
4     translate([0, 0, longitud])  
5         cylinder(r = radio * 1.5, h = 3);  
6 }  
7 tornillo(20);                // L=20, D=4  
8 tornillo(30, 6);             // L=30, D=6  
9 tornillo(diametro=5, longitud=15); // Argumentos nombrados
```

### 5.3 children() — Modulos que envuelven geometria

```
1 module envolvente(radio_borde = 2) {  
2     minkowski() {  
3         children(); // Toma lo que se pase entre llaves  
4         sphere(r = radio_borde);  
5     }  
6 }  
7 envolvente(3) {
```

```
8     cube([20, 20, 5], center=true);
9 }
```

## 5.4 DRY en CAD

```
1 // MAL: codigo duplicado
2 cube([10, 20, 5]);
3 translate([15, 0, 0]) cube([10, 20, 5]);
4
5 // BIEN: con modulo
6 module bloque() { cube([10, 20, 5]); }
7 bloque();
8 translate([15, 0, 0]) bloque();
```

## 5.5 Caso Real: Los modulos del SpiderBot

```
1 module soporte_servo_cadera() { ... } // bracket de cadera
2 module placa_base_cuadrupodo() { ... } // chasis inferior
3 module placa_superior() { ... } // chasis superior con cunas
4 module eslabon_completo() { ... } // femur completo
5 module tibia() { ... } // pierna inferior
6
7 // Composicion en ensamble_cuadrupodo.scad:
8 module pata_articulada(angulo_cadera=0, angulo_rodilla=0) {
9     dummy_servo();
10    rotate([0, angulo_cadera, 0])
11        eslabon_completo();
12    translate([55, 2.75, 0])
13        rotate([0, angulo_rodilla, 0])
14            tibia();
15 }
```

# Capítulo 6

## Iteracion y Simetria — El Bucle for

### 6.1 Sintaxis

```
1 for (i = [0 : 1 : 5]) {                               // Rango [inicio:incremento:fin]
2     translate([i * 10, 0, 0])
3     cube([5, 5, 5]);
4 }
5 for (i = [10, 20, 30, 50]) { ... } // Lista explicita
```

### 6.2 Distribucion radial

```
1 for (angulo = [0 : 60 : 300]) {
2     rotate([0, 0, angulo])
3     translate([40, 0, 0])
4     cylinder(r = 2, h = 10, center=true);
5 }
```

### 6.3 intersection\_for()

```
1 intersection_for(n = [0 : 5]) {
2     rotate([0, 0, n * 60])
3     translate([5, 0, 0])
4     cylinder(r = 3, h = 10);
5 }
```

### 6.4 List Comprehensions

```
1 cuadrados = [for (i = [0:10]) i * i];
2 pares = [for (i = [0:20]) if (i % 2 == 0) i];
```

## 6.5 Caso Real: Las 4 patas del SpiderBot

```
1 // Distribuye 4 soportes en 45, 135, 225, 315 grados
2 for (a = [45, 135, 225, 315]) {
3     rotate([0, 0, a])
4     translate([cuerpo_r - 7, 0, cuerpo_t/2])
5     rotate([0, 0, -90])
6     soporte_servo_cadera();
7 }
8
9 // Agujeros de pilares (0, 90, 180, 270 grados)
10 for (a = [0, 90, 180, 270]) {
11     rotate([0, 0, a])
12     translate([spacer_r, 0, 0])
13     cylinder(r=1.6, h=cuerpo_t+2, center=true);
14 }
15
16 // For anidado para pasacables
17 for (x = [-40, 40]) {
18     for (y = [-39, 39]) {
19         translate([x, y, 0])
20         cylinder(r=7.0, h=30, center=true);
21     }
22 }
```

# Capítulo 7

## Operaciones Avanzadas

### 7.1 hull() — Envolverte convexa

```
1 hull() {  
2     sphere(r = 10);  
3     translate([30, 0, 0])  
4         sphere(r = 8);  
5 }
```

### 7.2 minkowski() — Suma de Minkowski

```
1 // Redondear bordes de un cubo con una esfera  
2 minkowski() {  
3     cube([10, 10, 1]);  
4     cylinder(r=2, h=1);  
5 }
```

### 7.3 linear\_extrude() — Extrusion de 2D a 3D

```
1 linear_extrude(height = 10) { circle(r = 5); }  
2 linear_extrude(height = 20, twist = 90) { square([10, 10]); }
```

### 7.4 text() — Texto 2D

```
1 text("SpiderBot", size = 10);  
2 linear_extrude(height = 2)  
3     text("USS SpiderBot", size = 8, halign = "center");
```

## 7.5 offset() — Desplazamiento 2D

```

1 // Expandir con esquinas redondeadas
2 offset(r = 3) square([20, 20], center = true);
3 // Contraer y expandir para filete interno
4 offset(r = -2) offset(delta = 2)
5     square([20, 20], center = true);

```

## 7.6 projection() — Proyeccion 3D a 2D

```

1 projection(cut = false) sphere(r = 10); // Sombra
2 projection(cut = true) sphere(r = 10); // Corte transversal

```

## 7.7 polygon() — Perfiles 2D

```

1 polygon(points = [[0, 0], [10, 0], [5, 10]]);
2 linear_extrude(height = 3) {
3     polygon(points = [[0, 0], [20, 0], [0, 10]]);
4 }

```

## 7.8 rotate\_extrude() — Revolucion

```

1 rotate_extrude(angle = 360, convexity = 10) {
2     translate([20, 0, 0]) circle(r = 5);
3 }

```

## 7.9 Caso Real: El femur con hull()

```

1 union() {
2     hull() {
3         cylinder(r = 9, h = 6, center=true);
4         translate([link_len/2, -3, 1])
5             cube([12, 12, 8], center=true);
6     }
7     translate([link_len - 12, -8, 0])
8         cube([30, 28, link_t], center=true);
9 }

```

Gussets triangulares con polygon() + linear\_extrude():

```

1 translate([-22.25, cuerpo_r - 5.0, 5.5]) {
2     rotate([0, -90, 0])

```



```
3     linear_extrude(height = 2.0)
4         polygon(points = [[0, 0], [11, 0], [0, 5]]);
5 }
```

# Capítulo 8

## Archivos Multiples — use e include

### 8.1 use <archivo.scad>

Importa solo los modulos:

```
1 // main.scad
2 use <tornillo.scad>
3 tornillo(20); // El modulo esta disponible
```

### 8.2 import() — Importar STL/SVG/DXF

```
1 import("pieza_externa.stl");
2 linear_extrude(height = 3) import("logo.svg");
3 import("perfil.dxf");
```

### 8.3 include <archivo.scad>

Importa todo (menos usado):

```
1 include <tornillo.scad>
2 // Todo el codigo de tornillo.scad se ejecuta aqui
```

### 8.4 Organizacion de proyectos

```
robot/
+-- cuerpo.scad      // Chasis y soportes
+-- femur.scad       // Eslabon de pata
+-- tibia.scad       // Pierna inferior
+-- ensamble.scad    // Ensamble completo (solo use)
```

## 8.5 Caso Real: El ensamble del SpiderBot

```
1 // cad/ensamble_cuadruepodo.scad
2 use <cuero_cuadruepodo.scad>
3 use <eslabon_pata_cuadrupedo.scad>
4 use <tibia_pata.scad>
5 $fn= 20; // Prevalece para render mas rapido
6
7 // Archivos "instanciadores":
8 // placa_base_inferior.scad:
9 use <cuero_cuadruepodo.scad>
10 placa_base_cuadruepodo();
```

## Capítulo 9

# Renderizado, Debug y Exportacion a STL

### 9.1 Preview (F5) vs Render (F6)

| Aspecto        | Preview (F5)  | Render (F6)         |
|----------------|---------------|---------------------|
| Motor          | OpenCSG (GPU) | CGAL (CPU)          |
| Velocidad      | Instantaneo   | Puede tomar minutos |
| Precision      | Aproximada    | Exacta (manifold)   |
| Exportable STL | No            | Si                  |

### 9.2 Modifier Characters — Debug visual

| Caracter | Nombre     | Efecto                                  |
|----------|------------|-----------------------------------------|
| %        | Background | Gris translucido, excluido de booleanas |
| #        | Debug      | Rosa translucido, incluido              |
| !        | Root       | Solo este sub-arbol                     |
| *        | Disable    | Ignorado completamente                  |

```
1 difference() {
2     cube([30, 30, 10], center=true);
3     #translate([0, 0, -1])           // Rosa = debug
4         cylinder(r = 5, h = 12);
5     %translate([10, 0, -1])         // Gris = background
6         cylinder(r = 3, h = 12);
7     *translate([-10, 0, -1])        // Ignorado
8         cylinder(r = 3, h = 12);
9 }
```

## 9.3 render() — Forzar CGAL en preview

```

1 render(convexity = 4)
2     difference() {
3         cube([20, 20, 10], center=true);
4         sphere(r = 8);
5     }

```

## 9.4 \$t y animacion

Vista > Animacion > FPS=10, Steps=100:

```

1 translate([0, 0, 10 * sin($t * 360)]) sphere(r = 5);
2 rotate([0, 0, $t * 360]) cube([20, 5, 5], center=true);

```

## 9.5 \$preview — Diferenciar preview de render

```

1 $fpreview12 : 72;
2 cylinder(r = 10, h = 20);

```

## 9.6 Exportar a STL (F7)

Con F6 renderizado: Archivo > Exportar > Exportar como STL, o F7.

## 9.7 CLI de OpenSCAD

```

# Exportar pieza individual
openscad -o placa_base.stl cad/placa_base_inferior.scad
# Con parametros personalizados
openscad -o femur_largo.stl -D "link_len=70" cad/eslabon_pata_cuadripedo.scad
# Calidad especifica
openscad -o pieza.stl -D "$fn=100" archivo.scad

```

## 9.8 Parametros de calidad

| Parametro | Por defecto  | Recomendado STL |
|-----------|--------------|-----------------|
| \$fn      | (automatico) | 50–80           |
| \$fa      | 12 grados    | 6–2 grados      |
| \$fs      | 2mm          | 0.5–1mm         |

## 9.9 Caso Real: Comandos CLI para el SpiderBot

```
openscad -o placa_base_inferior.stl -D "$fn=60" cad/placa_base_inferior.scad
openscad -o femur.stl -D "$fn=50" cad/eslabon_pata_cuadrupeado.scad
openscad -o tibia.stl -D "$fn=50" cad/tibia_pata.scad
openscad -o placa_superior.stl -D "$fn=60" cad/placa_base_superior.scad
openscad -o spiderbot_ensamble.stl -D "$fn=30" cad/ensamble_cuadrupeado.scad
```

# Capítulo 10

## Proyecto Final — Replica el USS SpiderBot desde Cero

Ejercicio guiado. Las soluciones completas estan en cad/.

### 10.1 Paso 1: El chasis circular

Crea `mi_spiderbot.scad`:

1. Define `$fn = 50`.
2. Variables: `cuerpo_r = 65.0`, `cuerpo_t = 3.0`, `spacer_r = 55.0`.
3. Modulo `placa_base()` con `cylinder(r = cuerpo_r, h = cuerpo_t)`.
4. Con `difference()`, agrega 4 agujeros en 0, 90, 180, 270 grados a radio `spacer_r`.
5. Agrega 4 ranuras tipo velcro de `[4, 15, cuerpo_t+2]` en `(+-20, +-10)`.

### 10.2 Paso 2: El soporte de servo parametrico

Modulo `soporte_servo()`:

1. Base: `cube([36, 28, 15], center=true)` en `Z=7.5`.
2. Cavidad dual-depth: inferior a `Z=2.9`, superior a `Z=9.5`.
3. Ranura para bridas en `Y=5.5`.
4. Agujeros M2 con `rotate([90,0,0]) cylinder` en `X=+-14.25`.
5. Abertura delantera para corona en `(5.5, 12.0, 7.5)`.

## 10.3 Paso 3: Integrar soportes en el chasis

Dentro de `placa_base()`, agrega:

```
1 for (a = [45, 135, 225, 315]) {  
2     rotate([0, 0, a])  
3     translate([cuerpo_r - 7, 0, cuerpo_t/2])  
4     rotate([0, 0, -90])  
5     soporte_servo();  
6 }
```

## 10.4 Paso 4: El femur con hull()

Crea `femur.scad`. Define `link_len`, `link_h`, `link_t`. Usa `hull()` para conectar cilindro de cadera con bloque de rodilla.

## 10.5 Paso 5: La tibia

Crea `tibia.scad`. Define `tibia_h`, `tibia_t`. Usa tres `hull()` consecutivos. Agrega `sphere(r=5.5)` para el pie.

## 10.6 Paso 6: Ensamblar todo

Crea `ensamble_final.scad`:

1. use para importar tus 3 archivos.
2. Crea `dummy_servo()` y `pata_articulada()`.
3. `ensamble_completo()` con placa base, 4 patas y placa superior.

## 10.7 Paso 7: Exportar a STL

```
openscad -o mi_placa_base.stl -D "$fn=60" mi_spiderbot.scad  
openscad -o mi_femur.stl      -D "$fn=50" femur.scad  
openscad -o mi_tibia.stl      -D "$fn=50" tibia.scad
```



# Apéndice A: Hoja de Referencia Rápida

| Comando/Funcion  | Sintaxis                  | Descripcion               |
|------------------|---------------------------|---------------------------|
| cube             | cube([w,d,h], center)     | Cubo o prisma rectangular |
| sphere           | sphere(r)                 | Esfera                    |
| cylinder         | cylinder(r,h) / (r1,r2,h) | Cilindro o cono           |
| polyhedron       | polyhedron(points,faces)  | Poliedro arbitrario       |
| translate        | translate([x,y,z])        | Desplazar                 |
| rotate           | rotate([x,y,z])           | Rotar (grados)            |
| scale            | scale([x,y,z])            | Escalar                   |
| mirror           | mirror([x,y,z])           | Reflejo especular         |
| resize           | resize([x,y,z])           | Redimensionar exacto      |
| color            | color("nombre")           | Color (solo preview)      |
| union            | union() { }               | Fusionar solidos          |
| difference       | difference() { A; B; }    | Restar B de A             |
| intersection     | intersection() { }        | Interseccion              |
| hull             | hull() { }                | Envolvente convexa        |
| minkowski        | minkowski() { }           | Suma Minkowski            |
| linear_extrude   | linear_extrude(h)         | Extruir 2D a 3D           |
| rotate_extrude   | rotate_extrude()          | Revolucion 2D             |
| text             | text("str", size)         | Texto 2D                  |
| polygon          | polygon(points=[...])     | Poligono 2D               |
| offset           | offset(r) / delta         | Desplazar contorno 2D     |
| projection       | projection(cut)           | Proyectar 3D a 2D         |
| for              | for(i=[inicio:fin])       | Bucle (union)             |
| intersection_for | intersection_for(...)     | Bucle (interseccion)      |
| if               | if(cond){}else{}          | Condicional               |
| let              | let(v=val){}              | Variables locales         |
| assert           | assert(cond,"msg")        | Validacion                |
| module           | module nom(p){}           | Definir modulo            |
| function         | function nom(p)=expr      | Funcion (devuelve valor)  |
| children         | children(i)               | Hijos del modulo          |
| use              | use <archivo.scad>        | Importar modulos          |
| include          | include <archivo.scad>    | Importar todo             |
| import           | import("arch.stl")        | Importar STL/SVG/DXF      |

| Comando/Funcion | Sintaxis            | Descripcion             |
|-----------------|---------------------|-------------------------|
| echo            | echo(valor)         | Imprimir en consola     |
| str             | str(a,b,...)        | Concatenar a string     |
| concat          | concat(a,b,...)     | Concatenar listas       |
| len             | len(lista)          | Longitud                |
| [for]           | [for(i=[0:N]) expr] | List comprehension      |
| \$fn            | \$fn = N            | Fragmentos resolucion   |
| \$fa            | \$fa = ang          | Angulo minimo           |
| \$fs            | \$fs = tam          | Tamano minimo           |
| \$t             | \$t (lectura)       | Animacion (0 a 1)       |
| \$preview       | \$preview           | true en F5, false en F6 |
| render          | render(convexity=N) | Forzar malla CGAL       |
| %               | % { }               | Background (gris)       |
| #               | # { }               | Debug (rosa)            |
| !               | ! { }               | Root (solo esto)        |
| *               | * { }               | Disable (ignorar)       |

# Apéndice B: Recursos y Documentación Oficial

- Documentación oficial de OpenSCAD: <https://openscad.org/documentation.html>
- Cheat Sheet oficial: <https://openscad.org/cheatsheet/>
- Manual de usuario (Wikibooks): [https://en.wikibooks.org/wiki/OpenSCAD\\_User\\_Manual](https://en.wikibooks.org/wiki/OpenSCAD_User_Manual)
- Reddit: <https://reddit.com/r/openscad>
- GitHub: <https://github.com/openscad/openscad>
- Librería MCAD: <https://github.com/openscad/MCAD>
- dotSCAD (utilidades): <https://github.com/JustinSDK/dotSCAD>
- Repositorio USS SpiderBot: <https://github.com/moises-inc/USS-SPIDERBOT-solemne-3>

---

*Guía generada para el proyecto USS SpiderBot — Taller de Programación I, Universidad San Sebastian. 2026.*

*Los archivos de referencia están en **cad/** del repositorio.*